

Bonus 2: Attacco a un database MySQL

Report di Analisi di un Attacco SQL Injection con Wireshark

Studente: Leonardo Onori

Corso: Master Epicode in Cybersecurity and Ethical Hacking

Data: 4 Settembre 2026

1. Obiettivi

Analizzare, tramite Wireshark, un file di cattura di rete (SQL_Lab.pcap) contenente il traffico registrato durante un attacco di SQL injection precedentemente eseguito contro l'applicazione web vulnerabile DVWA (Damn Vulnerable Web Application), al fine di ricostruire le fasi dell'attacco e le informazioni sottratte dal database.

2. Contesto e scenario

Gli attacchi di SQL injection permettono a un aggressore di inserire istruzioni SQL arbitrarie all'interno di campi di input di un'applicazione web, ottenendo risposte dirette dal database sottostante. Questo consente di manomettere i dati, falsificare identità ed eseguire azioni dannose.

È stato fornito un file PCAP (SQL_Lab.pcap) contenente la cattura di un attacco reale contro un database MySQL, con l'obiettivo di visualizzarne le fasi tramite Wireshark e rispondere a una serie di domande di analisi.

3. Ambiente di lavoro

- Macchina virtuale: CyberOps Workstation (Linux)
- Strumento di analisi: Wireshark
- File analizzato: /home/analyst/lab.support.files/SQL_Lab.pcap
- Applicazione bersaglio: DVWA (Damn Vulnerable Web Application)
- Indirizzi coinvolti: 10.0.2.4 (host attaccante) → 10.0.2.15 (server vittima)
- Durata della cattura: 441 secondi (~8 minuti e 41 secondi)

4. Procedura ed evidenze

Parte 1 – Apertura del file PCAP e identificazione degli host

Il file SQL_Lab.pcap è stato aperto in Wireshark tramite File > Apri, dopo essere stato individuato nella directory /home/analyst/lab.support.files.

Domanda: quali sono i due indirizzi IP coinvolti nell'attacco?

Risposta: l'indirizzo 10.0.2.4 è l'host che genera le richieste (attaccante), mentre 10.0.2.15 è il server che ospita l'applicazione DVWA (vittima).

Parte 2 – Individuazione dell'inizio dell'attacco

Selezionando la riga 13 (richiesta GET HTTP) e utilizzando Segui > Flusso HTTP, si osserva la prima query di test dell'aggressore: `id=1%3D1` (equivalente a `1=1`), inserita nel campo UserID della pagina `/dvwa/vulnerabilities/sqli/`.

L'applicazione, invece di restituire un errore, risponde con un record valido (First name: admin, Surname: admin), confermando che l'input non è filtrato e che l'applicazione è vulnerabile alla SQL injection.

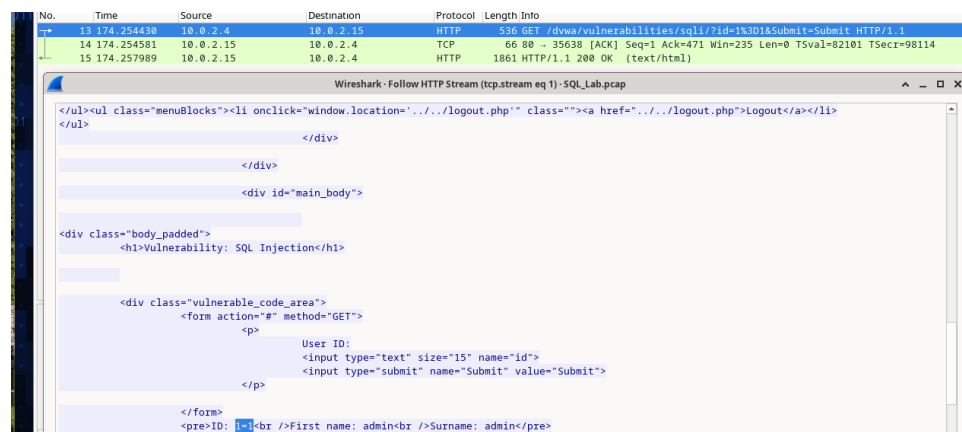


Fig. 1 – Flusso HTTP (stream 1): query `1=1` e risposta con record valido

Parte 3 – Prosecuzione dell'attacco: enumerazione di database e utente

Alla riga 19, l'aggressore invia la query: `1' or 1=1 union select database(), user()#`

La risposta del server rivela: `database = dvwa`, utente `database = root@localhost`. Vengono inoltre elencati più account applicativi (admin, Gordon Brown, Hack Me, Pablo Picasso, Bob Smith, dvwa).

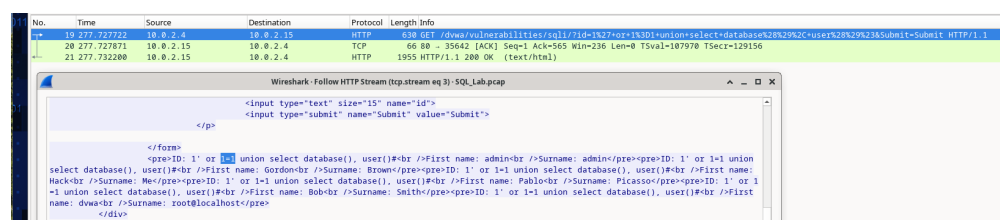


Fig. 2 – Flusso HTTP (stream 3): estrazione di database() e user()

Parte 4 – Informazioni di versione del sistema

Alla riga 22 viene inviata la query: 1' or 1=1 union select null, version()#

Domanda: qual è la versione?

Risposta: MySQL 5.7.12-0ubuntu1.1, valore visibile subito prima del tag di chiusura </pre></div> nella risposta HTML.

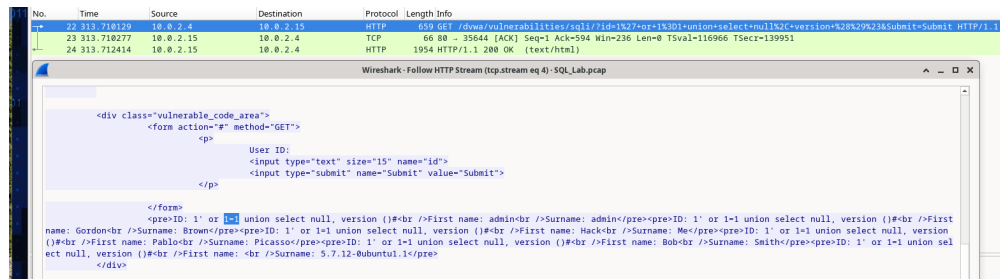


Fig. 3 – Flusso HTTP (stream 4): estrazione della versione del DBMS

Parte 5 – Enumerazione delle tabelle del database

Alla riga 25 viene inviata la query: 1' or 1=1 union select null, table_name from information_schema.tables#, che restituisce un lungo elenco di tabelle di sistema e applicative, tra cui la tabella users.

Domanda: cosa produrrebbe la query 1' OR 1=1 UNION SELECT null, column_name FROM INFORMATION_SCHEMA.columns WHERE table_name='users'?

Risposta: restituirebbe l'elenco dei nomi delle colonne della tabella users (es. user_id, first_name, last_name, user, password, avatar), un output molto più mirato rispetto all'elenco di tutte le tabelle del database, poiché filtrato sulla singola tabella di interesse.

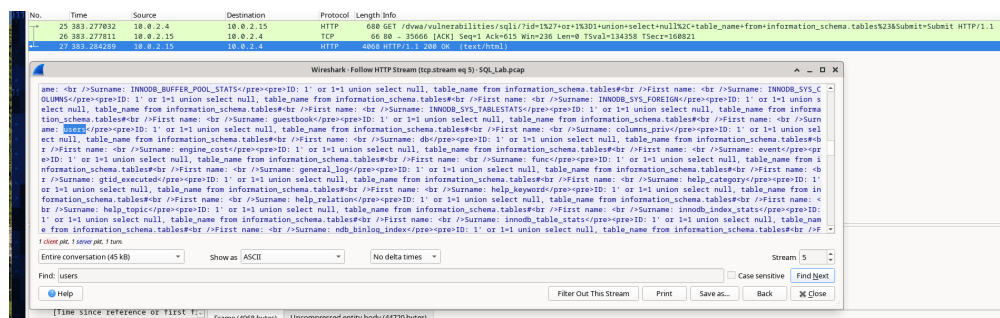


Fig. 4 – Flusso HTTP (stream 5): enumerazione delle tabelle, inclusa 'users'

Parte 6 – Estrazione di utenti e hash delle password

Alla riga 28 viene inviata la query conclusiva: `1' or 1=1 union select user, password from users#`, che estrae direttamente dalla tabella users le coppie utente/hash della password.

28	441.884078	10.0.2.4	10.0.2.15	HTTP	685	GET /dwa/vulnerabilities/sql/?id=1&27or=1&301=union+select+user&2C=password+from+users&23&Submit=Submit HTTP/1.1
29	441.884427	10.0.2.15	10.0.2.4	TCP	66	80 -> 35668 [ACK] Seq=1 Ack=620 Win=236 Len=0 TSval=148990 TSecr=178379
30	441.887286	10.0.2.15	10.0.2.4	HTTP	2091	HTTP/1.1 200 OK (text/html)

Fig. 5 – Pacchetti 28-30: richiesta GET finale e risposta HTTP 200 OK

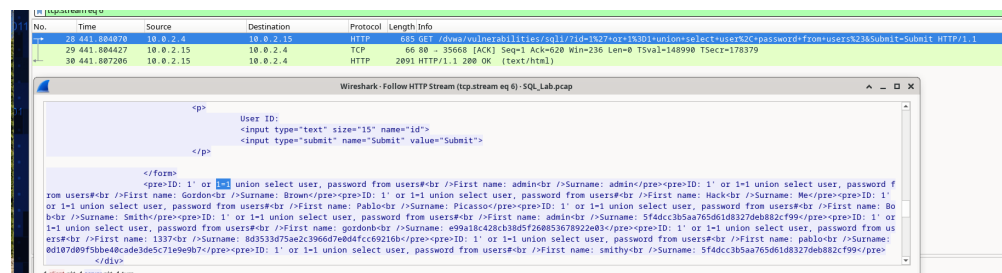


Fig. 6 – Flusso HTTP (stream 6): coppie utente/hash estratte dalla tabella users

Utente	Hash password (MD5)
admin	5f4dcc3b5aa765d61d8327deb882cf99
gordonb	e99a18c428cb38d5f260853678922e03
1337	8d3533d75ae2c3966d7e0d4fcc69216b
pablo	0d107d09f5bbe40cade3de5c71e9e9b7
smithy	5f4dcc3b5aa765d61d8327deb882cf99

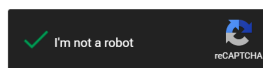
**Domanda: quale utente ha l'hash
8d3533d75ae2c3966d7e0d4fcc69216b?**

Risposta: l'utente 1337.

Domanda: qual è la password in chiaro?

Risposta: l'hash è stato decifrato tramite il servizio online CrackStation (crackstation.net), ottenendo la password in chiaro: charley.

Enter up to 20 non-salted hashes, one per line:



Crack Hashes

Fig. 7 – Inserimento dell'hash su CrackStation

Hash	Type	Result
8d3533d75ee2c3966d7e0d4fcc69216b	md5	charley

Color Codes: **Green**: Exact match, **Yellow**: Partial match — the hash column shows what that word really hashes to, with the **part yours matches** highlighted, **Red**: Not found.

Fig. 8 – Risultato: hash MD5 → password 'charley'

Nota: la coppia utente/password 1337/charley corrisponde a una delle credenziali di default note dell'applicazione DVWA, utilizzata a scopo didattico.

5. Domande di riflessione

1. Qual è il rischio che le piattaforme utilizzino il linguaggio SQL?

La maggior parte dei siti web si appoggia a database relazionali e al linguaggio SQL per gestire dati, autenticazione e contenuti dinamici. Se l'input fornito dall'utente non viene validato o filtrato correttamente, un aggressore può iniettare istruzioni SQL arbitrarie (SQL injection) all'interno delle query eseguite dall'applicazione. Questo, come dimostrato nel laboratorio, può portare alla lettura non autorizzata di dati riservati (nomi utente, hash delle password), alla scoperta della struttura del database e, potenzialmente, alla modifica o cancellazione di dati e al bypass dei meccanismi di autenticazione. La gravità dell'impatto dipende dai privilegi dell'account database utilizzato dall'applicazione e dalle intenzioni dell'aggressore.

2. Due metodi per prevenire attacchi di SQL injection:

- Query parametrizzate / prepared statement: separano nettamente il codice SQL dai dati forniti dall'utente, così che l'input venga sempre trattato come valore e mai come istruzione eseguibile.
- Validazione e sanitizzazione dell'input (whitelisting dei caratteri/formati ammessi), affiancata da un Web Application Firewall (WAF) in grado di rilevare e bloccare pattern tipici di SQL injection.

Ulteriori misure utili includono: applicazione del principio del privilegio minimo sull'account database, disabilitazione di funzionalità DBMS non necessarie e monitoraggio/logging delle istruzioni SQL eseguite.

6. Conclusioni

L'analisi del file SQL_Lab.pcap ha permesso di ricostruire, passo dopo passo, un attacco completo di SQL injection: dalla verifica iniziale della vulnerabilità (1=1), all'enumerazione di database e utente, alla scoperta della versione del DBMS, fino all'estrazione delle tabelle e, infine, delle credenziali degli utenti applicativi. L'esercizio evidenzia l'importanza critica della validazione dell'input lato server e dell'uso di query parametrizzate per prevenire questa categoria di vulnerabilità, ancora oggi tra le più diffuse e pericolose nelle applicazioni web (OWASP Top 10).